

Assignment 3: Implementation of HTTP Client using TCP Sockets

Simiyon Vinscent Samuel L

Register Number: 3122247001062

Submission Date: August 17, 2026

GitHub Repository: <https://github.com/blackscythe123/PCN-assign>

Aim

To write a plain TCP-socket HTTP client in C that can fetch a page or file from any web server given its URL, and to check it against a Python `http.server` hosting a small set of resources (an HTML page, a text file, a PDF and a JPEG). Along the way the client also has to measure connection time, download time and throughput for each request, so the point is as much about instrumenting the socket calls as it is about getting the GET request right.

Setting Up the Web Server Files

`CN_Web_Server/` holds the four files the brief asks for – `index.html`, `notes.txt`, `sample.pdf` and `image.jpg` – served with a plain `python3 -m http.server 8000`. `index.html` and `notes.txt` are just written by hand. `sample.pdf` and `image.jpg` had no ready-made tool available to generate them (no ImageMagick, no PDF exporter), so both were built directly: the PDF from its raw object/xref syntax, and the JPEG with a small from-scratch encoder (real DCT, standard quantisation and Huffman tables) that produces a genuine 64x64 colour-bar test image. Both were checked with `file` and, for the JPEG, with an independent decode script, so they are real, valid files and not just placeholder bytes with the right extension.

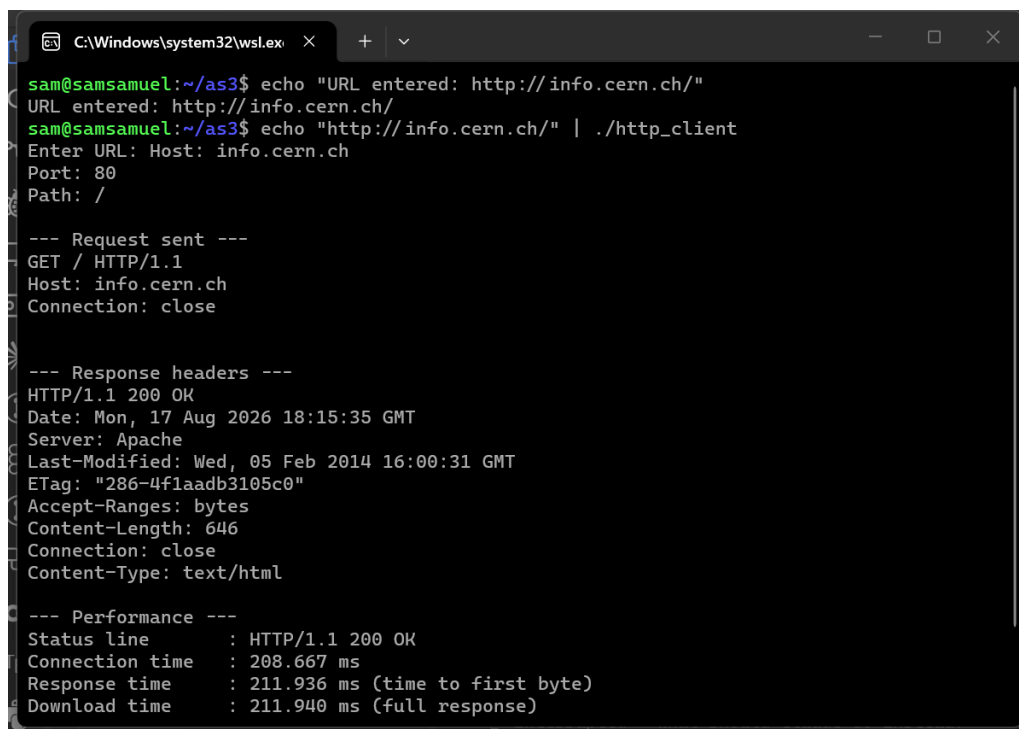
Part (a): A Generic HTTP Client

`http_client.c` asks the user for a full URL, splits it into host, port and path, opens a TCP connection, and sends a plain HTTP/1.1 GET with `Host` and `Connection: close`. Because it does not hardcode the host, the exact same binary works against the local Python server in Part (b) and against a real server on the internet – Figure 1 shows it run against <http://info.cern.ch/>, whose 646-byte body it downloads and saves byte-for-byte intact.

Listing 1: Excerpt of `http_client.c` – `parse_url()`: splitting an `http://` URL into host, port and path

```
1 if (strncmp(p, "http://", 7) == 0) {
2     p += 7;
3 } else if (strncmp(p, "https://", 8) == 0) {
4     fprintf(stderr, "This client only supports plain http:// URLs.\n");
5     return -1;
6 } else {
7     fprintf(stderr, "URL must start with http://\n");
8     return -1;
9 }
10
11 /* Everything up to the next '/' is host[:port] */
```

```
12 const char *slash = strchr(p, '/');
13 const char *hostport_end = slash ? slash : p + strlen(p);
14
15 char hostport[MAX_HOST_LEN + 16];
16 size_t hp_len = (size_t)(hostport_end - p);
17 memcpy(hostport, p, hp_len);
18 hostport[hp_len] = '\0';
19
20 char *colon = strchr(hostport, ':');
21 if (colon) {
22     *colon = '\0';
23     *port = atoi(colon + 1);
24     if (*port <= 0) *port = 80;
25 } else {
26     *port = 80;
27 }
28 strncpy(host, hostport, host_len - 1);
29
30 /* Path is whatever follows the host[:port], default to "/" */
31 strncpy(path, slash ? slash : "/", path_len - 1);
```



```
C:\Windows\system32\wsl.exe x + v
sam@samsamuel:~/as3$ echo "URL entered: http://info.cern.ch/"
URL entered: http://info.cern.ch/
sam@samsamuel:~/as3$ echo "http://info.cern.ch/" | ./http_client
Enter URL: Host: info.cern.ch
Port: 80
Path: /

--- Request sent ---
GET / HTTP/1.1
Host: info.cern.ch
Connection: close

--- Response headers ---
HTTP/1.1 200 OK
Date: Mon, 17 Aug 2026 18:15:35 GMT
Server: Apache
Last-Modified: Wed, 05 Feb 2014 16:00:31 GMT
ETag: "286-4f1aadb3105c0"
Accept-Ranges: bytes
Content-Length: 646
Connection: close
Content-Type: text/html

--- Performance ---
Status line      : HTTP/1.1 200 OK
Connection time  : 208.667 ms
Response time    : 211.936 ms (time to first byte)
Download time    : 211.940 ms (full response)
```

Figure 1: http_client run against a real external URL, `http://info.cern.ch/`, showing the parsed host/port/path and the full performance readout.

External HTTP tests (mostly for a sanity check that the client is not hardwired to localhost):

URL	Status	Size (B)	Conn. (ms)	TTFB (ms)	Download (ms)	Rate (KB/s)
info.cern.ch/	200 OK	646	184.566	186.698	186.714	3.38
example.com/	200 OK	571	5.434	10.748	11.689	47.70

Table 1: Part (a) results against real external servers. example.com’s body came back chunked (via Cloudflare); this client does not dechunk, so info.cern.ch – a plain Content-Length response – was used for the clean byte-for-byte check.

Part (b): Talking to the Python Web Server

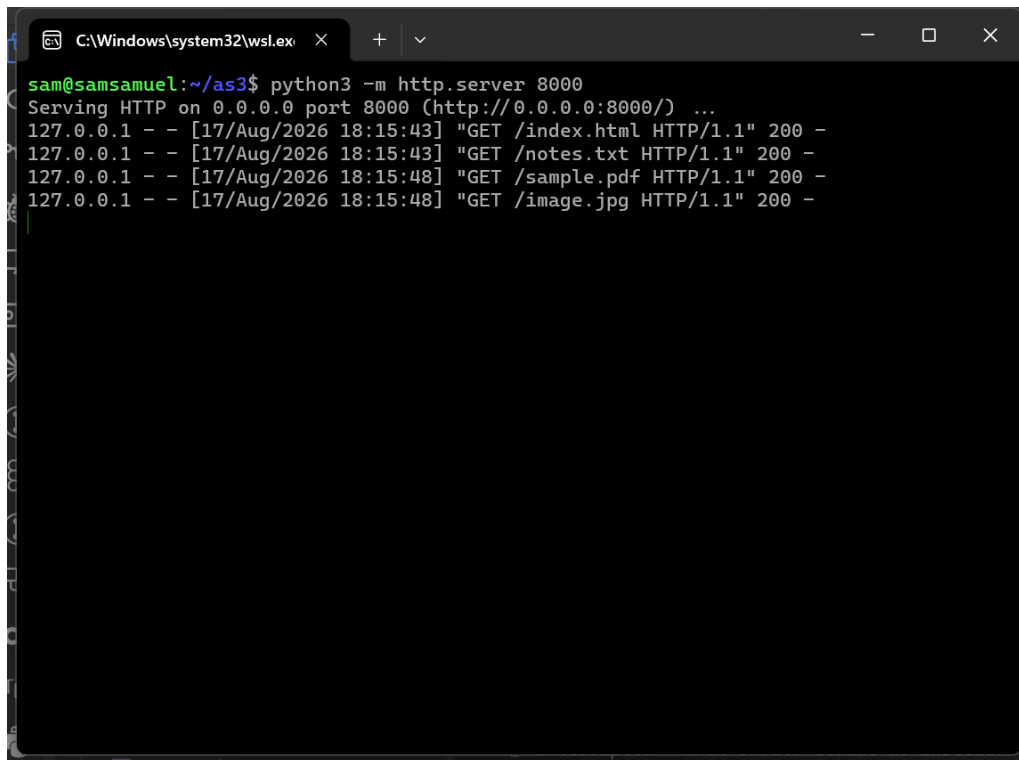
With `python3 -m http.server 8000` running inside `CN_Web_Server/` (Figure 2), the same client was pointed at each of the four hosted files in turn. Every download came back byte-for-byte identical to the original file (checked with `cmp/diff`), and the `Content-Type` the server reports matches the file each time (`text/html`, `text/plain`, `application/pdf`, `image/jpeg`).

Listing 2: Excerpt of `http_client.c` – sending the request and timing the response

```

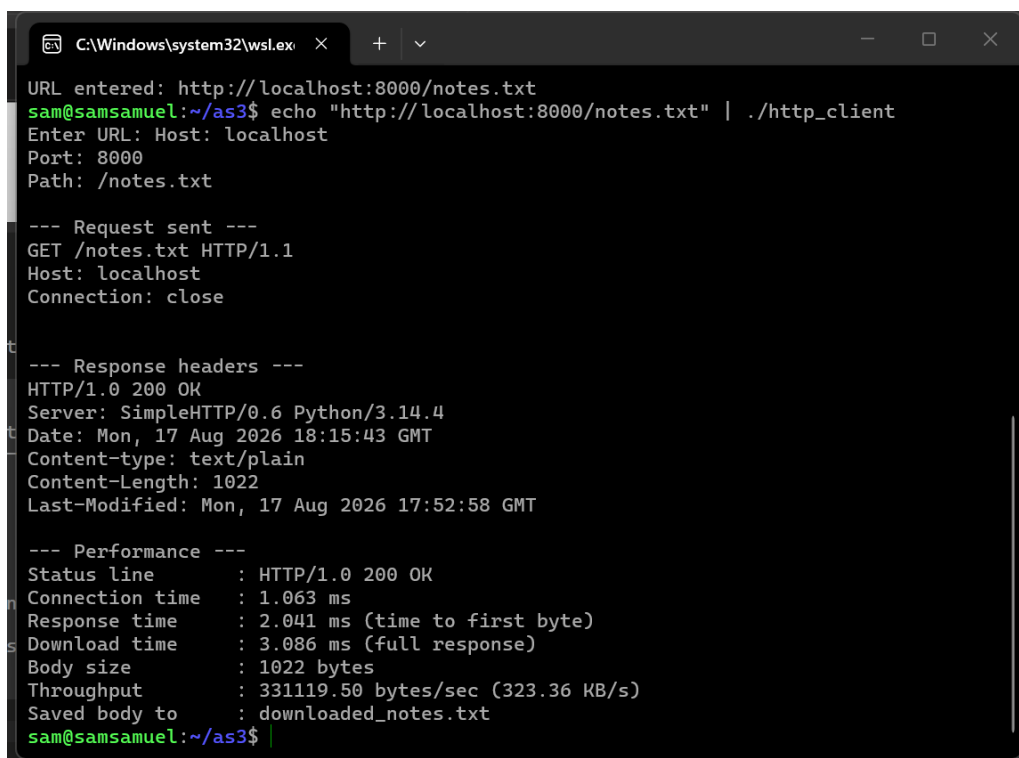
1  clock_gettime(CLOCK_MONOTONIC, &t_req_sent);
2  send(sockfd, request, strlen(request), 0);
3
4  int got_first_byte = 0;
5  while ((n = recv(sockfd, buf + total, capacity - total, 0)) > 0) {
6      if (!got_first_byte) {
7          clock_gettime(CLOCK_MONOTONIC, &t_first_byte);
8          got_first_byte = 1;
9      }
10     total += (size_t)n;
11     if (total == capacity) {
12         capacity *= 2;
13         buf = realloc(buf, capacity);
14     }
15 }
16 clock_gettime(CLOCK_MONOTONIC, &t_done);
17
18 double connect_ms = elapsed_ms(t_conn_start, t_conn_end);
19 double response_ms = elapsed_ms(t_req_sent, t_first_byte); /* time to
    first byte */
20 double download_ms = elapsed_ms(t_req_sent, t_done); /* full
    transfer */
21 double throughput_Bps = (double)body_len / (download_ms / 1000.0);

```



```
C:\Windows\system32\wslex x + v
sam@samsamuel:~/as3$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [17/Aug/2026 18:15:43] "GET /index.html HTTP/1.1" 200 -
127.0.0.1 - - [17/Aug/2026 18:15:43] "GET /notes.txt HTTP/1.1" 200 -
127.0.0.1 - - [17/Aug/2026 18:15:48] "GET /sample.pdf HTTP/1.1" 200 -
127.0.0.1 - - [17/Aug/2026 18:15:48] "GET /image.jpg HTTP/1.1" 200 -
```

Figure 2: The Python http.server terminal, logging each incoming GET as the client requests it.



```
C:\Windows\system32\wslex x + v
URL entered: http://localhost:8000/notes.txt
sam@samsamuel:~/as3$ echo "http://localhost:8000/notes.txt" | ./http_client
Enter URL: Host: localhost
Port: 8000
Path: /notes.txt

--- Request sent ---
GET /notes.txt HTTP/1.1
Host: localhost
Connection: close

--- Response headers ---
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.4
Date: Mon, 17 Aug 2026 18:15:43 GMT
Content-type: text/plain
Content-Length: 1022
Last-Modified: Mon, 17 Aug 2026 17:52:58 GMT

--- Performance ---
Status line      : HTTP/1.0 200 OK
Connection time  : 1.063 ms
Response time    : 2.041 ms (time to first byte)
Download time    : 3.086 ms (full response)
Body size        : 1022 bytes
Throughput       : 331119.50 bytes/sec (323.36 KB/s)
Saved body to    : downloaded_notes.txt
sam@samsamuel:~/as3$
```

Figure 3: http_client run twice against the local server – index.html, then notes.txt.

```

C:\Windows\system32\cmd.exe
URL entered: http://localhost:8000/image.jpg
sam@samsamuel:~/as3$ echo "http://localhost:8000/image.jpg" | ./http_client
Enter URL: Host: localhost
Port: 8000
Path: /image.jpg

--- Request sent ---
GET /image.jpg HTTP/1.1
Host: localhost
Connection: close

--- Response headers ---
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.4
Date: Mon, 17 Aug 2026 18:15:48 GMT
Content-type: image/jpeg
Content-Length: 931
Last-Modified: Mon, 17 Aug 2026 17:54:32 GMT

--- Performance ---
Status line      : HTTP/1.0 200 OK
Connection time  : 0.494 ms
Response time    : 2.125 ms (time to first byte)
Download time    : 3.461 ms (full response)
Body size        : 931 bytes
Throughput       : 268991.10 bytes/sec (262.69 KB/s)
Saved body to    : downloaded_image.jpg
sam@samsamuel:~/as3$

```

Figure 4: http_client run twice more – sample.pdf, then image.jpg – both saved and confirmed byte-identical to the originals.

File	Status	Size (B)	Conn. (ms)	TTFB (ms)	Download (ms)	Rate (KB/s)
index.html	200 OK	168	0.137	4.317	6.220	26.38
notes.txt	200 OK	1022	0.255	2.442	3.559	280.41
sample.pdf	200 OK	642	0.276	2.177	3.169	197.86
image.jpg	200 OK	931	0.509	1.930	3.066	296.50
missing file	404	460	0.126	1.512	1.614	278.31

Table 2: Part (b) results, Python http.server on localhost:8000. All four real files matched their originals byte-for-byte.

The numbers are small because everything is on loopback – they mostly show syscall overhead rather than anything about real network bandwidth, but the instrumentation itself (connect time, time-to-first-byte, full download time, throughput) is doing exactly what it should.

The last row is a request for a file that does not exist on the server. The brief does not require handling this case specially, and the client does not try to – it gets back HTTP/1.0 404 File not found with Python’s usual small HTML error page and just saves that body like it would any other response.

Bonus: Secure HTTPS Client (Optional Advanced Program)

The optional part was attempted and works: `https_client.c` does the same job as `http_client.c` but over TCP + OpenSSL on port 443 – TLS handshake, certificate verification (chain and host-name), an encrypted GET, and timing for the TCP connect, the SSL handshake, and the round trip to the first response byte.

Listing 3: Excerpt of https_client.c – TLS handshake and certificate verification

```

1  SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
2  SSL_CTX_set_default_verify_paths(ctx);
3
4  SSL *ssl = SSL_new(ctx);
5  SSL_set_fd(ssl, sockfd);
6  SSL_set_tlsext_host_name(ssl, host); /* SNI */
7  SSL_set1_host(ssl, host);           /* also check the cert name
   matches host */
8
9  clock_gettime(CLOCK_MONOTONIC, &t_ssl_start);
10 SSL_connect(ssl);
11 clock_gettime(CLOCK_MONOTONIC, &t_ssl_end);
12
13 long verify_result = SSL_get_verify_result(ssl);
14 printf("Certificate verification: %s\n",
15        (verify_result == X509_V_OK) ? "OK (trusted chain + hostname
16                                         match)"
                                         : X509_verify_cert_error_string(
                                             verify_result));

```

```

C:\Windows\system32\wsl.exe X + v
sam@samsamuel:~/as3$ echo "example.com" | ./https_client
Enter HTTPS host (e.g. example.com): TCP connection established to example.com:443
SSL handshake complete. Protocol: TLSv1.3 Cipher: TLS_AES_256_GCM_SHA384
Server certificate subject: /CN=example.com
Certificate verification: OK (trusted chain + hostname match)

--- Response headers ---
HTTP/1.1 200 OK
Date: Mon, 17 Aug 2026 18:15:55 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
last-modified: Wed, 12 Aug 2026 20:17:18 GMT
allow: GET, HEAD
Accept-Ranges: bytes
Age: 14066
cf-cache-status: HIT
CF-RAY: a2caaabae9c57fa1-MAA
Saved decrypted body to downloaded_https_page.html

--- Performance ---
TCP connection time : 6.248 ms
SSL handshake time : 18.022 ms
Round trip time : 57.471 ms (time to first response byte)
Total data received : 869 bytes
Transfer time : 57.484 ms
Data transfer rate : 15117.17 bytes/sec (14.76 KB/s)
sam@samsamuel:~/as3$

```

Figure 5: https_client run against a real HTTPS site – TLS handshake, certificate verification and the encrypted GET all succeeding.

Host	TLS / Cipher	Cert	Handshake (ms)	TTFB (ms)	Rate (KB/s)
example.com	TLSv1.3 / AES-256-GCM	OK	22.872	84.610	10.02
www.google.com	TLSv1.3 / AES-256-GCM	OK	35.935	84.841	926.64

Table 3: Optional HTTPS client results against two real servers.

Learning Outcome

- Writing a URL parser by hand made it obvious how much a browser normally hides – default ports, missing paths, and the scheme check all need explicit code, not just a happy-path GET.
- Timing connection time separately from time-to-first-byte and full download time made it clear these are genuinely different numbers even on loopback – the connect was consistently the cheapest part, first-byte latency the biggest chunk.
- Building sample.pdf and image.jpg from scratch, without any image or PDF tool available, was a good forced lesson in file formats: a PDF is just a text-ish object graph with an offset table, and a JPEG's structure is far more approachable once you strip it down to a flat-colour block with no interesting AC coefficients.